# TermXT scripting language user manual

Current version: **1.0.3**.

TermXT runs `.xt` scripts inside xTerminal. A script can use variables, conditions, loops, functions, file operations and xTerminal commands. This edition covers runtime error locations and live editor diagnostics. The 1.0.2 PDF and 1.0.0 PDF remain available as older references.

## Contents

Running and checking scripts . . . . . 2

Editor diagnostics and line display . . . . . 2

Arguments and variables . . . . . 4

Text and arithmetic . . . . . 5

Conditions . . . . . 6

Loops . . . . . 7

Functions . . . . . 7

Commands, pipes and errors . . . . . 8

Input and files . . . . . 9

Examples and changes in 1.0.3 . . . . . 10

# Running and checking scripts

```
xt script.xt
xt "scripts with spaces\report.xt" -p production 8080
xt -new script.xt
xt -edit script.xt
xt -check script.xt
xt -ver
xt -h
```

`-new` writes a template to the specified file, overwriting an existing file. `-edit` opens the built-in editor; `xte script.xt` also opens it. Scripts are checked for structural errors before execution. `-check` checks without executing commands and reports failure through xTerminal's command error state. Structural checks include unmatched blocks, invalid branch ordering, missing assignments, empty variable names, duplicate functions and misplaced `break`, `continue` or `return`. Each diagnostic uses `Line N: message`. Any structural error prevents execution and produces a `Script not executed` summary. These checks happen before the script runs, so a `try/catch` inside the script cannot handle them.

A successful check does not guarantee that expressions, commands or file paths will succeed at runtime. The editor adds live errors and warnings beyond the structural checks performed by `xt -check`.

## Editor suggestions

In `xte script.xt`, enter insert mode with `i` or `Insert`. IntelliSense suggests TermXT keywords after two characters, built-in functions at the start of a `set` value, condition operators, and declared functions after `call .` Terminal command suggestions are available after `run`, in `capture` assignments and after pipes. Typing `{` opens variable suggestions, including inside quoted text. These include built-in variables and names declared by `set`, `input`, `read`, `capture` and `each`. Variables are also suggested after `lines:` in an `each` loop. Suggestions ignore case and comments.

Use **Ctrl+Space** to open suggestions manually, **Up/Down** to select an item, and **Enter** or **Tab** to accept it. Variable completion supplies the closing `}` or reuses an existing one. **Esc** closes the popup; press it again to return to normal mode. The popup shows usage hints beside each suggestion.

# Editor diagnostics and line display

Open a script with `xte script.xt` or `xt -edit script.xt`. Errors and warnings update as you edit, including unsaved changes. Save with `:w` before running `xt -check script.xt` or `xt script.xt`, which read the file on disk.

The left gutter displays source line numbers. A red `!` marks a line with an error; a yellow `?` marks a warning. If a line has both, the error marker takes priority. Wrapped continuation rows show `+` rather than a new source line number. Wrapping therefore does not change the line numbers in diagnostics. The header shows counts such as `[E:1] [W:1]`.

Press **F2** for the complete diagnostic list. Entries identify severity, source line and column, a diagnostic code when available, and a description. For example, `L3:C8 XT1003` means line 3, column 8, diagnostic XT1003. Both positions are 1-based. Use **Up/Down** to select, **PgUp/PgDn** or the mouse wheel to scroll, and **Enter** to jump to the selected location. **F2** or **Esc** closes the list. Fixed issues disappear from the markers, counts and open list as the buffer updates.

In normal mode, type `:` followed by a command and press **Enter**:

| Command | Action |
|---|---|
| `:errors` | Show a summary of errors. |
| `:warnings` | Show a summary of warnings. |
| `:next-error` | Jump to the next error. |
| `:next-warning` | Jump to the next warning. |

## Errors and warnings

Structural errors use the shared block and statement checks. The editor also reports these additional diagnostics:

| Code | Meaning |
|---|---|
| `XT1001` | Invalid condition syntax, such as an unclosed quote or parenthesis, or a missing operand. |
| `XT1002` | Invalid `each` syntax or a numeric range endpoint outside the 32-bit integer range. |
| `XT1003` | Missing or unknown function name, or malformed quoted arguments to `call`. |
| `XT1004` | A constant `loop` count is not a positive 32-bit integer. |
| `XT1005` | An invalid constant `eval`, `substr` or `replace` expression. |
| `XT1006` | Missing or malformed arguments to `write` or `append`. |
| `XT2001` | A referenced variable has no declaration in the script. |
| `XT2002` | Assignment to `DATE`, `TIME` or `CWD`; interpolation computes these values dynamically. |
| `XT2003` | Trailing arguments ignored by a keyword such as `end`, `try`, `catch` or `exit`. |
| `XT2004` | A constant `wait` value has no effect; use a positive integer number of milliseconds. |
| `XT2005` | A likely misspelled TermXT keyword. |

`XT1001` through `XT1006` are errors; `XT2001` through `XT2005` are advisory warnings. The editor does not execute commands or resolve every runtime value. Checks that need a constant value are deferred when that value contains a variable placeholder. An undeclared-variable warning does not change the runtime rule that unknown placeholders remain literal text. A likely keyword typo is a warning because an ordinary line can be an external command.

### Reading a diagnostic

This intentionally incorrect script produces one warning and one error:

```
set name = Ada
  print "{nmae}"
  call missing
```

The header shows `[E:1] [W:1]`. The F2 list includes:

```
[WARNING] 1/2  L2:C10 XT2001: Variable 'nmae' has no declaration in this script.
[ERROR] 2/2  L3:C8 XT1003: Function 'missing' not found.
```

Change `{nmae}` to `{name}` and define the requested function or call an existing one. Select an entry and press **Enter** to move directly to the affected text.

## Arguments and variables

Arguments after `-p` become `{1}`, `{2}`, and so on. Double quotes keep an argument containing spaces together. `""` passes an empty argument, which still counts toward `{argc}`. A quoted `-p` inside a path or argument is literal text. Spaces and tabs can separate arguments and language keywords.

```
# xt greet.xt -p "Ada Lovelace"
set name = reader
if {argc} >= 1
    set name = {1}
end
print "Hello {name}"
```

Use `{argc}` to test whether an argument was supplied. Missing variables stay literal: an undefined `{1}` produces the text `{1}`. Comparing `{1}` with itself, or with `null`, does not detect a missing argument. `null` is ordinary text.

```
set name = Ada
set message = "Hello {name}"
print "{message}"
```

Variable names and language keywords are case-insensitive. Names consist of word characters (letters, digits and underscores). Values are text; `set` creates or replaces a variable. Variables other than positional arguments are shared across function calls. Lines beginning with `#`, after optional indentation, are comments. Blank lines are ignored. Comments occupy their own lines.

| Variable | Value |
|---|---|
| `{1}`, `{2}`, ... | Arguments of the current script or function call. |
| `{argc}` | Number of arguments in the current script or function call. |
| `{DATE}` | Current date, `yyyy-MM-dd`. |
| `{TIME}` | Current time, `HH:mm:ss`. |

| Variable | Value |
|---|---|
| `{USER}` | Current account name. |
| `{PC}` | Computer name. |
| `{CWD}` | Current xTerminal working directory. |
| `{i}` | Current loop iteration, starting at 1. Nested loops restore the outer counter. |
| `{result}` | Most recent return value; cleared when entering a function call. |
| `{error}` | `true` following a failed operation and on entry to `catch`; `false` after a successful command or file operation, or a successfully handled `try/catch`. |
| `{error_message}` | Most recent runtime error, including `[line N]`; available inside `catch` and after a reported error that allows execution to continue. |

`DATE`, `TIME` and `CWD` are evaluated when used. Interpolation is a single pass: placeholder-shaped text inside a substituted value is not expanded again.

# Text and arithmetic

```
set total = eval (10 + 2) * 3
set remainder = eval 10 % 3
set loud = upper hello world
set quiet = lower HELLO WORLD
set size = len "hello world"
set clean = trim "  hello  "
set part = substr "hello world" 6 5
set changed = replace "hello world" "world" "TermXT"
print "{total}: {changed}"
```

`eval` supports numbers, decimal points, `+`, `-`, `*`, `/`, `%` and parentheses. Results use a decimal point regardless of the Windows number format. Invalid expressions and non-finite results, including division by zero, report an error and enter an enclosing `catch`. Outside a `try`, failed arithmetic reports the error and supplies `0`; inspect `{error}` before using that value.

`substr` uses a zero-based start and a nonnegative length. Negative starts are clamped to zero, lengths are clamped to the remaining text, and a start beyond the text returns an empty string. Negative lengths report an error. `replace` matches text without regard to case. Quote text arguments that contain spaces; use `""` for an empty replacement.

`print` adds a newline. It recognizes `\n`, `\t` and `\\` escape sequences, including those stored in variables. Absolute Windows drive paths and UNC paths are preserved by its path handling.

# Conditions

```
if {count}>=2&&{count}<5
    print "Between two and four"
elif {count} == 5
    print "Exactly five"
else
    print "Outside the range"
end

if not {error} && ({mode} == quick || {mode} == full)
    print "Ready"
end

if "ready||waiting" contains "||"
    print "The operator is part of the text"
end
```

| Operator | Meaning |
|---|---|
| `==`, `!=` | Case-insensitive text equality or inequality. |
| `>`, `<`, `>=`, `<=` | Numeric comparison when both values parse as numbers; otherwise case-insensitive text ordering. |
| `contains` | Left text includes right text. |
| `startswith`, `endswith` | Left text has the specified prefix or suffix. |
| `not` | Negates the following comparison or grouped condition. |
| `&&`, `\|\|` | Logical AND and OR, with short-circuit evaluation. |

Precedence is parentheses, comparisons/`not`, `&&`, then `||`. For example, `not false && false` is false; use `not (false && false)` to negate the whole expression. Symbolic operators can touch their operands; word operators such as `contains` require whitespace around them. Conditions can be nested up to the interpreter's evaluation depth limit of 128.

Operators and parentheses inside double-quoted text are literal. Substituted variable values are treated as data, even if they contain operators, quotes or newlines. Undefined placeholders retain their literal value. For a truthy test, empty text, `0` and `false` are false; other text is true. Equality is textual, so `1` and `1.0` are different for `==`. Write an empty comparison operand as `""`.

Each `if` allows multiple `elif` branches, followed by at most one `else`. Neither `elif` nor another `else` can follow `else`.

# Loops

```
loop 3
    print "Iteration {i}"
end

set remaining = 3
while {remaining} > 0
    print "{remaining}"
    set remaining = eval {remaining} - 1
end

each color in red,green,blue
    print "{i}: {color}"
end

each n in 3..1
    print "{n}"
end

read report = "report.txt"
each line in lines:report
    print "{line}"
end
```

`loop` requires a positive integer. Numeric `each` ranges are inclusive, support ascending and descending order, and accept signed 32-bit endpoints. A range at either integer limit terminates without wrapping. Comma-separated iteration skips empty items and trims surrounding whitespace; it is not a CSV parser for quoted commas. `lines:report` and `lines:{report}` iterate nonempty lines of a variable, trimming each line.

`break` exits the innermost loop; `continue` skips to its next iteration. `return` inside a loop exits its function. A function cannot use `break` or `continue` to control a caller's loop. An inner loop restores the outer `{i}`, including when an error leaves the inner loop. `wait 1000` pauses for one second. `exit` stops the entire script, including when used in a function.

# Functions

```
func greet
    return Hello {1}
end

set person = "Ada Lovelace"
call greet {person}
print "{result}"
```

Functions may be called before their definition. Function names are case-insensitive and duplicate definitions are rejected. `call` parses arguments before substituting variables, so `{person}` remains one argument when its value contains whitespace or quotes. Literal multiword arguments still need quotes.

Each call has its own positional arguments and `{argc}`. Caller arguments are restored on return and on failure, without a 20-argument limit. New arguments do not leak into the caller. Other variables remain shared. `return value` stores text in `{result}` and ends the function; bare `return` stores empty text. Call nesting is limited to 128 to prevent runaway recursion from crashing xTerminal.

# Commands, pipes and errors

```
run time
capture files = ls
capture selected = ls | cat -s exe
print "{selected}"

try
    read settings = "settings.txt"
    print "Loaded settings"
catch
    print "Could not load settings: {error_message}"
end
print "Continuing"
```

Ordinary xTerminal commands can also be written without `run`. `capture` stores standard output, trimming its surrounding whitespace. A single | outside double quotes separates pipeline stages. Quoted pipes remain part of an argument; || is not split into pipeline stages.

## Handling failures

Inside `try`, the first failed script operation, command failure flag or thrown exception stops that block and enters `catch`. Later pipeline stages also stop after a failure. `{error}` is `true` on entry to the handler and `{error_message}` describes the failure, including its source line in the form `[line 7] message`. If the handler succeeds, execution continues after `end` with `{error}` set to `false`. A handler failure propagates to an enclosing handler and identifies the new failing line inside `catch`. A `try` without a `catch` propagates a failure to an enclosing handler. If no handler exists, it reports the error once, skips the remaining try body and continues after `end` with the error state set.

Starting a `try` clears the previous error state. Successful commands and file operations clear it as well. Simple statements such as `print` do not clear a previous failure. A successful handler resets `{error}` at its end, but does not necessarily clear `{error_message}`: a handler that only prints the message leaves it available. A successful command or file operation, or a new `try`, clears the message. Save it to another variable before those operations if needed.

Outside `try`, reported script errors set `{error}` and the xTerminal command error flag and may allow the next statement to run. For example, failed `eval` reports an error and supplies `0`. Unhandled exceptions, such as a failed file write or malformed runtime condition, stop execution. Use `try/catch` when later statements must be skipped after either kind of failure.

## Runtime line numbers

Runtime errors include `[line N]` in the console message and in `{error_message}`. Numbers refer to the 1-based source line in the `.xt` file, including blank lines and comments. They identify the failing statement inside a function or nested block, even after control returns to a caller. A failing `elif` or a repeated `while` condition points to that condition's own line. The runtime prefix is `[line N]`; structural checks use `Line N:` and the editor uses locations such as `L3:C8`.

For `run`, bare commands and `capture`, a command failure flag produces `[line N] Command failed: command text`. A thrown exception carries its own message with the same line prefix. A failed pipeline points to the script line that invoked the pipeline, and later stages are skipped.

Save this example exactly as shown. The blank second line counts:

```
# Error location demo

try
    call missing
    print "This line is skipped"
catch
    print "Caught: {error_message}"
end
print "After catch: {error}"
```

With no function named `missing`, the output is:

```
  [line 4] Function 'missing' not found.
Caught: [line 4] Function 'missing' not found.
After catch: false
```

The first line is the interpreter's report. The second is explicitly printed by the handler, so seeing the same location there is expected. Propagating an already reported failure through a `try` without `catch` does not add another automatic report. The editor also flags this deliberate missing call as XT1003.

# Input and files

```
input name = "Your name"
write "output report.txt" "Hello {name}"
append "output report.txt" "Generated on {DATE} at {TIME}"
read report = "output report.txt"
print "{report}"
```

Paths are relative to xTerminal's current working directory, unless absolute. Use double quotes for paths with spaces. `read` loads the whole file. `write` overwrites or creates a file; `append` adds to it or creates it. Both add a newline after the supplied text. `write "empty line.txt" ""` writes one empty line. Parent directories must already exist. File-write text supports interpolation and escape sequences in the script literal; substituted file contents are preserved.

# Examples and changes in 1.0.3

The example directory contains `greet.xt`, `countdown.xt`, `portcheck.xt`, `sysreport.xt`, `netaudit.xt`, and `language_features.xt`.

```
xt Documents\TermXT_Examples\language_features.xt -p "Ada Lovelace"
```

This local demo creates `TermXT demo output.txt` in the current directory and demonstrates optional arguments, functions, compact/grouped conditions, quoted paths and recovery from a deliberate arithmetic error.

## New in 1.0.3

Runtime error messages now carry `[line N]` in console output and `{error_message}`. Locations are preserved through functions, loops, nested handlers and re-evaluated conditions. Errors in a handler identify the handler's failing line. A `try` without a handler keeps the original location and avoids reporting the same failure twice automatically.

The editor now displays live TermXT errors and warnings from unsaved changes, with source line markers, header counts and a navigable F2 list. It checks conditions, function calls and constant expressions/arguments, and warns about undeclared variables, ignored assignments or arguments, ineffective waits and likely keyword typos. Suggestions also cover conditions and terminal commands.

When updating scripts that log `{error_message}`, remember that the value includes the source line prefix. Save the message before a successful command or file operation clears it. Use F2 to inspect live issues, then save and run `xt -check` before execution. A successful structural check does not include all of the editor's additional diagnostics.

## Earlier changes in 1.0.2

Version 1.0.2 added `{argc}`, grouped conditions and compact symbolic condition operators. It fixed quoted logical text crashes, `not` precedence, repeated condition interpolation, tab-separated keywords, quoted `-p` parsing, function argument leakage, exception cleanup, stale return values at call entry, missed `try/catch` failures, quoted file paths, nested iteration counters and integer range overflow. Structural validation is shared with the editor. The countdown, port checker and network audit examples now use `{argc}` for their defaults.

When updating older scripts, use `{argc}` for optional arguments, put empty comparison operands in quotes, and use parentheses if `not` should negate a compound condition. Function-call argument substitution now preserves each variable value as one argument.